

BigBurger AI

Enterprise Multi-Tenant Document Intelligence Platform

Upgrading from Regional Silos to Unified Knowledge

Lead Engineer Design Proposal April 2026 - GCP-Native Architecture

The Challenge: From Silos to Synthesis

Current State:

- 4 independent regional subsidiaries (North, South, East, West)
- Each runs its own Google Drive with PDFs + scanned images
- A single-tenant PoC works for scoped regional questions

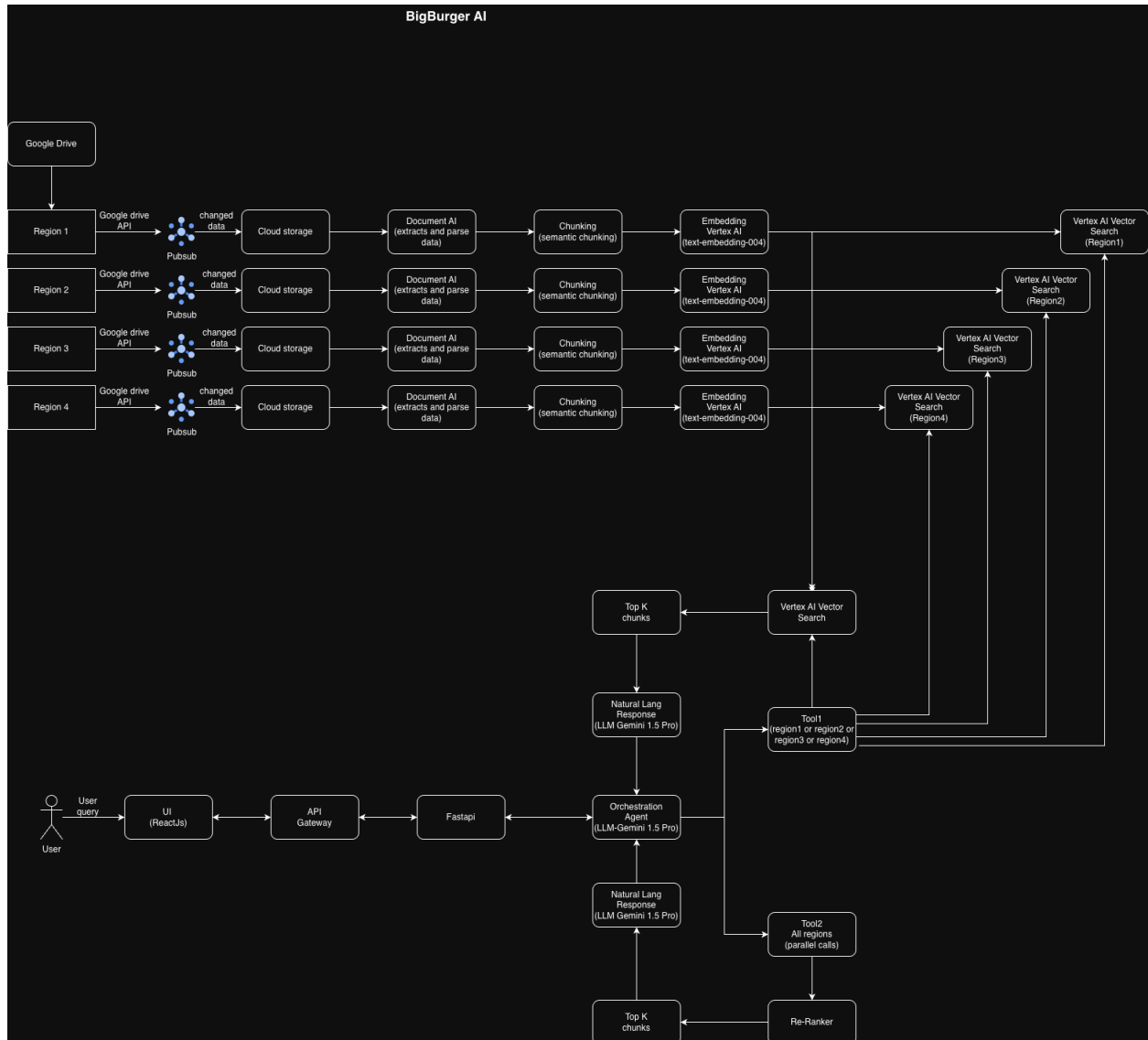
Target State:

"Which subsidiaries have conflicting exclusivity clauses with our national beverage supplier, and what is our total aggregated liability?"

Three hard problems this introduces:

1. **Multi-tenancy** — data from 4 siloed Drive environments must be queryable together without data bleed
2. **Cross-regional synthesis** — an LLM must reason across all subsidiaries in one coherent answer
3. **Access control** — regional managers see only their region; global auditors see all.

High-Level System Architecture (Image)



Pillar 1: Data Ingestion Pipeline

Triggering: Event-Driven Change Detection

- **Google Drive API** webhooks push change notifications to a **Pub/Sub** topic per region
- Cloud Run Job subscribes to the topic and downloads the changed file to **Cloud Storage** (tenant-prefixed bucket: gs://bigburger-docs/north/, etc.)
- No polling — near-real-time sync with sub-minute latency

OCR & Extraction

- **Document AI** processes scanned PDFs and images → extracts clean text
- Form Parser extracts structured fields (dates, parties, amounts) for metadata enrichment

Chunking Strategy

- **Semantic chunking** using RecursiveCharacterTextSplitter at sentence/paragraph boundaries
 - Chunk size: ~400–600 tokens; overlap: ~80 tokens
 - Preserves legal clause boundaries (critical for accurate retrieval)
- Each chunk carries full provenance metadata: doc_id, region, subsidiary, doc_type, expiry_date, liability_amount, version

Embedding

- **Vertex AI text-embedding-004** (768 dimensions) — GCP-native, no external API dependency
- Embeddings upserted to **Vertex AI Vector Search** index (one index per region = tenant isolation)

Pillar 1 (cont.): Sync, Versioning & Deletions

Document Versioning — Firestore as the Registry

Firestore collection: document_registry/{region}/{doc_id}

```
{
  doc_id:    "north-bev-001",
  region:    "north",
  drive_file_id: "1aB2cD...",
  current_version: 3,
  ingested_at: "2026-04-01T10:00:00Z",
  gcs_path:   "gs://bigburger-docs/north/north-bev-001_v3.pdf",
  status:     "active" // or "deleted"
}
```

Update Workflow (when a Drive file is modified)

1. Pub/Sub message received → version bump in Firestore
2. New file version downloaded to Cloud Storage
3. Re-chunked and re-embedded → chunks upserted by {doc_id}_{chunk_index} ID
4. Orphaned chunks from previous version deleted from Vector Search using doc_id metadata filter

Deletion Workflow

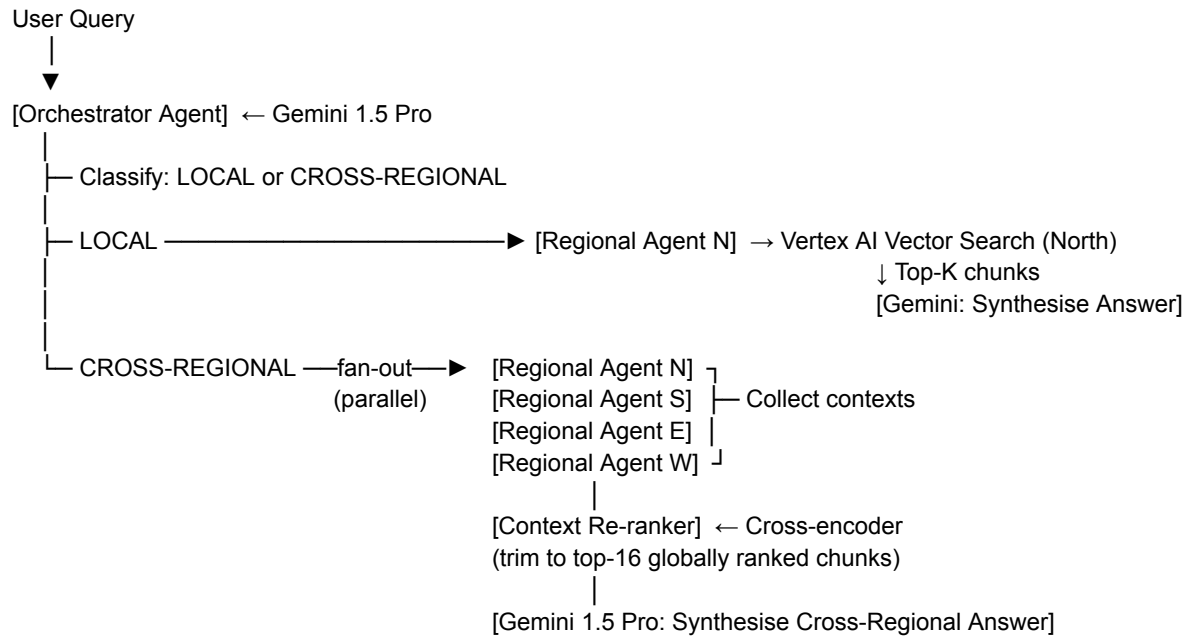
1. Drive deletion event received → Firestore status set to "deleted" (soft-delete)
2. Vector Search bulk-delete by doc_id metadata filter
3. 30-day tombstone retention for audit trail before hard-delete

Conflict / Race Condition Handling

- Firestore optimistic locking on current_version prevents double-ingestion of the same file version

Pillar 2: AI & Agentic Architecture

Two-Tier Agent Design (Google ADK)



Context-Window Exhaustion Prevention

- 4 regions × 4 chunks × ~500 tokens ≈ **8,000 tokens** — far below Gemini's 1M limit
- Cross-encoder re-ranker trims to top-16 chunks when corpus grows
- Structured prompt with explicit region tags prevents hallucination / region confusion

Pillar 2 (cont.): Token Cost, Rate Limits & Latency

Token Cost Optimization

Query Type	Model	Approx. Cost per Query
Classification (local vs cross-regional)	Gemini 2.0 Flash	~\$0.0001
Local retrieval + synthesis	Gemini 2.0 Flash	~\$0.001
Cross-regional synthesis	Gemini 1.5 Pro	~\$0.005

- Use the **smallest capable model** for each step (Flash for classification, Pro for complex synthesis)
- Cache frequent queries in **Memorystore (Redis)** — 1-hour TTL for identical query+user-role combinations
- Reduce re-embedding cost by caching query embeddings

Rate Limit Handling

- Vertex AI Embedding API: wrap in **Cloud Tasks queue** with rate limiting (requests/min configurable)
- Gemini API: exponential backoff + jitter on 429 responses; queue depth monitored in Cloud Monitoring
- Regional fan-out uses `asyncio.gather` — parallel calls, not sequential, keeping latency $\approx$ single-region latency

Latency Budget (P95 target: < 4 seconds)

Query classification: ~200ms (Gemini Flash)
Parallel regional retrieval: ~500ms (Vector Search ANN)
Context re-ranking: ~300ms (cross-encoder, skipped for simple queries)
LLM synthesis: ~1,500ms (Gemini Pro)

Total: ~2,500ms ✓

Pillar 3: Security & Access Control

Authentication (AuthN)

- **Identity Platform** (Firebase Auth) issues signed JWTs
- All API calls require a valid Authorization: Bearer <token> header
- Backend (Cloud Run) validates the JWT signature on every request using `firebase_admin.auth.verify_id_token()`
- Tokens expire after 1 hour; refresh tokens managed client-side

Authorization (AuthZ) — RBAC with Custom JWT Claims

// JWT custom claims for a regional manager:

```
{ "role": "regional_manager", "allowed_regions": ["north"] }
```

// JWT custom claims for a global auditor:

```
{ "role": "global_auditor", "allowed_regions": ["north", "south", "east", "west"] }
```

Defence-in-Depth (two enforcement layers)

Layer 1: Query-time RBAC (Orchestrator Agent)

→ Filter requested regions against JWT `allowed_regions` before ANY retrieval

Layer 2: Storage-layer isolation (Vertex AI Vector Search)

→ Each region is a separate index — a mis-scoped query physically cannot return data from another index

Additional Controls

Control	GCP Service
Data perimeter	VPC Service Controls
Encryption at rest	Cloud KMS (CMEK per region)
Audit logging	Cloud Audit Logs (all data-read events)
Secret management	Secret Manager (API keys, service account keys)
Network isolation	VPC with Private Service Connect to Vertex AI

Pillar 4: Observability & Evaluation

Pre-Deployment Accuracy Testing

Golden Dataset (Ground Truth)

- Legal team curates 50–100 QA pairs per region: {question, expected_answer, source_doc_id}
- Separate "conflict detection" test set: 20 cross-regional edge cases including known conflicts

RAG Evaluation Metrics (using Vertex AI Evaluation or Ragas)

Metric	Target	Description
Faithfulness	> 0.90	Answer claims are grounded in retrieved context
Answer Relevance	> 0.85	Answer addresses the question asked
Context Precision	> 0.80	Retrieved chunks are actually relevant
Context Recall	> 0.80	Relevant docs are found in the retrieved set
Conflict Detection Rate	> 0.95	Cross-regional conflicts are correctly identified

Regression Suite

- Run full golden dataset after any model update, embedding change, or chunking strategy change
- CI/CD gate: deployment blocked if any metric drops > 5% from baseline

Pillar 4 (cont.): Production Monitoring & Feedback Loop

Production Monitoring (Cloud Monitoring + Cloud Logging)

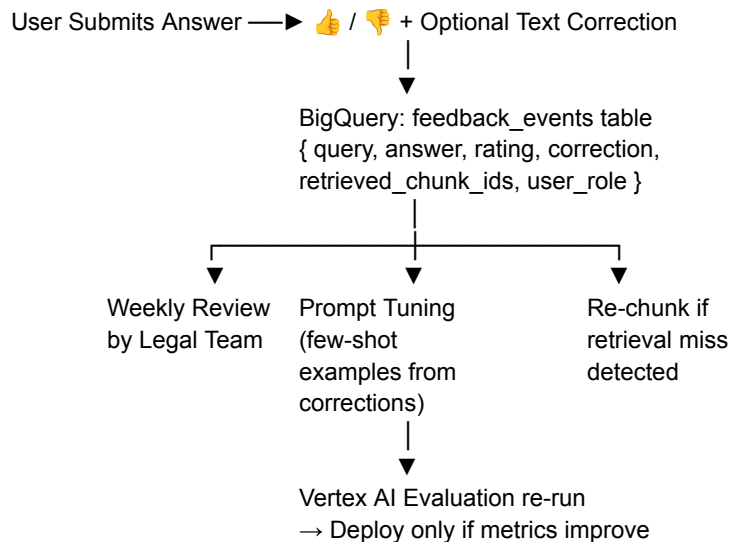
Custom Metrics (emitted from Cloud Run):

- bigburger/query_latency_ms — P50, P95, P99 by query_type, user_role
- bigburger/retrieval_score — avg cosine similarity of top-K results
- bigburger/llm_faithfulness — sampled automated faithfulness scoring
- bigburger/auth_rejection_rate — spikes may indicate breach attempts

Alerting:

- P95 latency > 5s → PagerDuty alert (SLA breach)
- Retrieval score drops > 10% → Slack alert (embedding drift or index corruption)
- Auth rejection rate > 5% in 5 min → Security alert

Continuous Improvement Feedback Loop



Design Trade-offs

Decision	Choice Made	Alternative Considered	Rationale
Vector DB	Vertex AI Vector Search (one index/region)	Single index with metadata namespace filter	Hard tenant isolation at storage layer; simpler RBAC; slightly higher infra cost
LLM	Gemini 1.5 Pro (synthesis) + Flash (classify)	GPT-4o / Claude	GCP-native = no cross-cloud data egress; tighter IAM integration
Embedding Model	Vertex AI text-embedding-004	OpenAI text-embedding-3-large	Same GCP-native rationale; 768-dim is sufficient for contract retrieval
Chunking	Semantic / recursive (400–600 tokens, 80 overlap)	Fixed-size (512 tokens)	Legal contracts have clause boundaries; semantic chunking preserves them
Agent Pattern	Supervisor + Sub-Agent (ADK multi-agent)	Single monolithic RAG chain	Fan-out parallelism; each regional agent independently testable; scales to N regions
Data Sync	Pub/Sub webhook + Firestore version ledger	Nightly batch job	Near-real-time update (<1 min vs overnight stale lag)
OCR	Document AI	Tesseract (open-source)	Production-grade accuracy on scanned legal docs; GCP-managed; no self-hosted infra
Auth	Identity Platform + custom JWT claims	Cognito / Auth0	GCP-native; single vendor for identity + audit; integrates with Cloud IAM
Cross-encoder re-ranker	Optional (enabled at >8K token threshold)	Always on / always off	Cost vs quality balance; cheap Flash model can skip re-ranking for simple queries

Prototype Overview & What Was Built

Code Prototype: `prototype/`

Demonstrates the hardest part of this problem — **Multi-Tenant Routing & Retrieval** — with:

`prototype/`

```
├── config.py                # RBAC roles, mock users (simulates JWT claims)
├── mock_data/contracts.json # 16 realistic contracts across 4 regions
├── ingestion/
│   ├── chunker.py          # Word-overlap chunker (semantic boundary-aware)
│   └── pipeline.py         # Load → chunk → embed → index orchestration
├── retrieval/
│   ├── vector_store.py     # ChromaDB multi-tenant store (per-region collections)
│   └── retriever.py        # RBAC-enforced retriever
├── agents/
│   ├── rbac.py             # authenticate() + authorize_region_access()
│   ├── regional_agent.py   # Single-region scoped retrieval agent
│   └── orchestrator_agent.py # Query classifier + cross-regional fan-out + synthesis
└── main.py                 # 7 live demo scenarios
```

Key Demo Scenarios

1. **Local query** — regional manager, single region, beef contract expiry
2. **Cross-regional conflict detection** — global auditor finds East/BubbleDrink conflict (\$1.2M)
3. **RBAC enforcement** — regional manager silently scoped to their region only
4. **Auth rejection** — unknown user ID blocked at authentication layer
5. **Compliance deadlines** — multi-region health inspection schedule query
6. **East NYC certificate expiry** — regional manager, local compliance query
7. **Cross-regional lease liability** — HQ executive aggregated lease obligations

Run it in 3 commands:

```
cd prototype && pip install -r requirements.txt && python main.py
```